```
    </body>
</html>
```

Let us understand this code a bit. The first part of the code is purely the HTML code for the input textbox, the *Predict* button and the output section. The 'script' portion is where the TensorFlow.js magic works. It just creates a very simple, linear regression sort of one-layer sequential dense model.

```
const model=tf.sequential();
                model.add(
                    tf.layers.dense({
                        units:1,
                        inputShape:[1],
                        bias: true
                    })
                );
```

The model is built with SDG (stochastic descent gradient) as optimiser, MSE (mean square error) as loss function and the accuracy measurement yardstick. This code looks very similar to the familiar Python Keras code.

```
                model.compile({
                    loss:'meanSquaredError',
                    optimizer: 'sgd',
                    metrics: ['mse']
                });
```

Next, we need to get the training data set ready. We are training with 1 to 10 as input (x values) and input * 3 -1 as output (y values):

```
                const train_labels = tf.tensor1d([1, 2, 3, 4,
5, 6, 7, 8, 9, 10]);
                const train_output = tf.tensor1d([2, 5, 8, 12,
14, 18, 21, 23, 26, 29]);
```

Now, let's do some training with the 'fit' method of TensorFlow layer's API:

```
await model.fit(train_labels, train_output, {epochs:100});
```

Next, let's have the prediction, by getting the input text the user has given. Note the TensorFlow.Js layer's 'model.predict' API (there's no change when compared to Python Keras!):

```
const inputVal = parseInt(document.
getElementById('inputNumber').value);
document.getElementById('prediction').innerText +=
                                    model.predict(tf.
tensor1d([inputVal])).dataSync();
```

In the end, the '*do_prediction()*' method is invoked on clicking the *Predict* button:

```
    <button onclick="do_prediction()">Predict</button>
```

And the output in your favourite browser is something like what's shown in Figure 1.

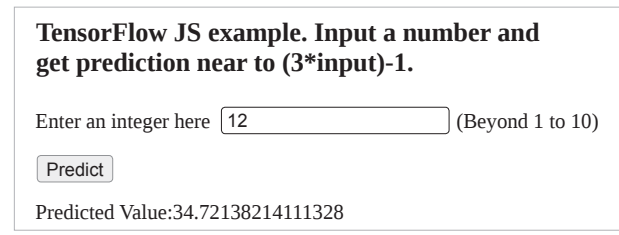

Figure 1: The demo program output

Takeaway

It is an accepted fact today that the fully connected neural network based inference works perfectly in browsers. Typically, it's not the training but the inference that happens in the browser. Keeping in mind the fact that ConvNetJS is not maintained any more, Google's TensorFlow.js is the best bet in this regard. It is not necessary that the browser's host must be GPU accelerated, because the sort of inference that runs on a browser does not fully exploit the GPU. Hence, running it in a 'normal laptop' is good enough as far as the inference task in the browser is concerned. END

References

[1] Moving Deep Learning into Web Browser: How Far Can We Go? Ma, Xiang, Zheng, Tian, Lie et. al. *https://arxiv.org/abs/1901.09388*
[2] TensorFlow JS, *https://github.com/tensorflow/tfjs*
[3] Keras JS, *https://github.com/transcranial/keras-js*
[4] WebDNN, *https://github.com/mil-tokyo/webdnn*
[5] ConvNetJS, *https://cs.stanford.edu/people/karpathy/convnetjs/*
[6] Brain JS, *https://github.com/BrainJS*
[7] Mind, *https://github.com/stevenmiller888/mind*
[8] Synaptic, *https://github.com/cazala/synaptic*
[9] TensorFlow.js model converter, *https://github.com/tensorflow/tfjs/tree/master/tfjs-converter)*
[10] MNIST digit detection using TensorFlow, *https://blog.pragmatists.com/machine-learning-in-the-browser-with-tensorflow-js-2f941a8130f5*

By: Pradip Mukhopadhyay

The author has 20 years of experience across the stack, ranging from low level system programming to high level GUI. He is a FOSS enthusiast and is currently working for NetApp, Bengaluru.